Thomas RIPOLL
Bintou DIALLO
M1 Data Engineering et IA

Cahier de charges

Thématique : Optimisation des trajets et de la rentabilité des flottes de taxi new-yorkais face aux contraintes tarifaires et logistiques.

- Problématiques 1 : Analyse financière des performances des chauffeurs de taxi. (Par exemple : les chauffeurs sont-ils mieux payés en fonction de quartier, les taxes ont-elles un impact sur leur revenu, etc)

- Problématique 2 : Analyse de la fluidité du trafic en fonction du trajet et des horaires.

- Problématique 3 : Prédiction de la demande en taxis, selon l'horaire et la zone géographique/le trajet (Solution de Machine Learning en fonction du flux de passagers et les quartiers)

—----------

1. Nos datasets

2 datasets trouvées : Nous nous sommes basées sur les données de taxis New Yorkais données sur cette plateforme officielle de la ville de New York :

https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

Nous avons choisi de nous focaliser sur le mois de février 2025 pour nos analyses, suffisamment riche en données pour que nous puissions réaliser notre projet.

Premier dataset trouvé :

yellow_tripdata_2025-02.parquet :

- Ce fichier contient des données sur les trajets des taxis new yorkais allant d'informations précises sur leurs facturation (y compris les pourboires, taxes, etc) , les taxis, l'horaire de début/de fin du trajet, le quartier, etc

- C'est un fichier en format parquet, ce qui est idéal pour nous étant donné qu'il est déjà en format adéquat pour le Big Data.

Il contient les colonnes suivantes :

- **VendorID :** Un code qui indique le “TPEP provider”, donc le fournisseur du terminal du taxi (ce sont eux qui permettent d’avoir la ligne de données)

- **tpep_pickup_datetime :** Date et heure où le terminal a commencé à enregistrer une course (= début de trajet)

- **tpep_dropoff_datetime :** Date et heure où le terminal a arrêté d’enregistrer la course (= fin de trajet)

- **passenger_count :** Nombre de passagers dans le véhicule.

- **trip_distance :** distance en miles parcouru lors du trajet

- **RatecodeID :** Le code du tarif final appliqué à la fin du trajet (ex: 1 = Standard, 2 = JFK, etc.)

- **store_and_fwd_flag :** indique si l'enregistrement a été conservé dans la mémoire du véhicule avant l'envoi au serveur (Y/N)

- **PULocationID :** Lieu où le terminal a commencé à enregistrer une course (= lieu du début de trajet)

- **DOLocationID :** Lieu où le terminal a arrêté d’enregistrer la course (= lieu de fin de trajet)

- **payment_type :** Le code numérique indiquant comment le passager a payé (1 = carte de crédit, 2 = espèces, etc.)

- **fare_amount :** Le tarif de base de la course calculé par le compteur

- **extra :** valeur des charges supplémentaires ajoutées au tarif de base de la course

- **mta_tax :** Une taxe imposée par la MTA (Metropolitan Transportation Authority), qui est déclenchée automatiquement en fonction du tarif utilisé

- **tip_amount :** montant du pourobire versée pour la course

- **tolls_amount :** montant payé pour les péages durant la course

- **improvement_surcharge :**
  (pour plus d’infos :
  https://www.nyc.gov/site/tlc/about/improvement-surcharge-payment-dates.page)

- **total_amount :** montant total facturé au passager

- **congestion_surcharge :** Le montant total perçu durant le trajet pour la taxe de congestion de l'État de New York (NYS)

- **airport_fee :** Les frais facturés spécifiquement pour les départs depuis les aéroports LaGuardia et JFK.

- **cbd_congestion_fee :** Une redevance forfaitaire appliquée par trajet pour la zone de réduction de la congestion de la MTA

Deuxième dataset trouvé :

taxi_zone_lookup :

- Ce fichier contient des données sur les zones des taxis (qu'ils peuvent être amenés à fréquenter ou traverser)

- C'est un fichier CSV (on devra en changer le type pour qu'il soit compatible avec du Big Data

Il contient les colonnes suivantes :

- **LocationID :** Identifiant unique numérique de la zone géographique (clé de jointure)

- **Borough :** Arrondissement de New York dans lequel se trouve la zone

- **Zone :** Le nom précis du quartier ou de la sous-zone

- **service_zone :** La catégorie de zone de service (ex: Yellow Zone, Boro Zone)

Troisième dataset (que nous allons créer) :
Afin de réaliser des jointures par la suite (nous détaillerons nos jointures dans la prochaine partie), nous allons créer une table par nous même en nous basant sur un document que nous avons reçues sur le site :

le fichier est le suivant : **data_dictionary_trip_records_yellow.pdf**

voici un aperçu :

Field Name	Description
VendorID	A code indicating the TPEP provider that provided the record. 1 = Creative Mobile Technologies, LLC 2 = Curb Mobility, LLC 6 = Myle Technologies Inc 7 = Helix
tpep_pickup_datetime	The date and time when the meter was engaged.
tpep_dropoff_datetime	The date and time when the meter was disengaged.
passenger_count	The number of passengers in the vehicle.
trip_distance	The elapsed trip distance in miles reported by the taximeter.
RatecodeID	The final rate code in effect at the end of the trip. 1 = Standard rate 2 = JFK 3 = Newark 4 = Nassau or Westchester 5 = Negotiated fare 6 = Group ride 99 = Null/unknown
store_and_fwd_flag	This flag indicates whether the trip record was held in vehicle memory before sending to the vendor, aka "store and forward," because the vehicle did not have a connection to the server. Y = store and forward trip N = not a store and forward trip
PULocationID	TLC Taxi Zone in which the taximeter was engaged.
DOLocationID	TLC Taxi Zone in which the taximeter was disengaged.
payment_type	A numeric code signifying how the passenger paid for the trip. 0 = Flex Fare trip 1 = Credit card 2 = Cash 3 = No charge 4 = Dispute 5 = Unknown 6 = Voided trip
fare_amount	The time-and-distance fare calculated by the meter. For additional information on the following columns, see https://www.nyc.gov/site/tlc/passengers/taxi-fare.page
extra	Miscellaneous extras and surcharges.
mta_tax	Tax that is automatically triggered based on the metered rate in use.
tip_amount	Tip amount – This field is automatically populated for credit card tips. Cash tips are not included.
tolls_amount	Total amount of all tolls paid in trip.
improvement_surcharge	Improvement surcharge assessed trips at the flag drop. The improvement surcharge began being levied in 2015.
total_amount	The total amount charged to passengers. Does not include cash tips.
congestion_surcharge	Total amount collected in trip for NYS congestion surcharge.
airport_fee	For pick up only at LaGuardia and John F. Kennedy Airports.
cbd_congestion_fee	Per-trip charge for MTA's Congestion Relief Zone starting Jan. 5, 2025.

Ce fichier indique les correspondances entre les valeurs qui vont être indiquées et leur valeur métier.

Par exemple pour un attribut “payment_type”, une valeur 1 signifie que le type de paiement utilisé est une carte de crédit.

Ainsi, nous allons créer nous même une table qui indiquera ces correspondances, en vue de faire de futures jointures pour que les métiers qui accèderont aux données que nous leur fournissons aient une vision plus claire de ce à quoi cela correspond.

2. Les jointures que l’on souhaite réaliser :

1. Jointures Géographiques (Dataset 1 + Dataset 2) :

Nous allons faire une jointure entre notre premier dataset (yellow_tripdata_2025-02.parquet) et notre deuxième dataset (taxi_zone_lookup) afin de pouvoir récupérer les noms réels des quartiers

- Jointure de la zone de départ : Jointure entre la colonne PULocationID de notre dataset principal et la colonne LocationID du dataset taxi_zone_lookup.
- Jointure de la zone d'arrivée : Jointure entre la colonne DOLocationID de notre dataset principal et la colonne LocationID du dataset taxi_zone_lookup.

2. Jointures Sémantiques (Dataset 1 + Dataset 3 que nous créons) :

Nous allons faire une jointure entre les données de notre dataset 1 (yellow_tripdata_2025-02.parquet) et celle de la base que nous allons créée (celle basée sur data_dictionary_trip_records_yellow.pdf, ce sera une table de référence) afin de pouvoir rendre la donnée plus compréhensible pour les équipes métiers à qui nous allons destiner chacun de nos data marts. En effet, il sera plus simple pour eux de savoir que le moyen de paiement pour une transaction correspond à “carte bancaire” plutôt qu’à 1. Nous réaliserons les jointures suivantes :

- Jointure sur le moyen de paiement : Jointure entre la colonne payment_type de notre dataset principal et l'identifiant de notre table de référence des paiements (ex: pour transformer “1” en “Credit card”).

- Jointure sur le fournisseur (Vendor) : Jointure entre la colonne VendorID de notre dataset principal et notre table de référence des fournisseurs (ex: pour transformer “2” en “Curb Mobility”).

- Jointure sur le type de tarification : Jointure entre la colonne RatecodeID de notre dataset principal et notre table de référence des tarifs (ex: pour transformer “2” en “JFK”).

3. Choix d’architecture

Architecture médaillon :

Raw - Couche Bronze :

- Stockage des données brutes sur un Datalake (HDFS).
- **Toutes les données seront strictement stockées au format Parquet** (optimisation du stockage en colonnes, compression et lectures analytiques rapides).

Couche Silver :

- Utilisation de PySpark pour le nettoyage, les filtres et les jointures.
- Les données standardisées vont former notre Data Warehouse et seront gérées via **HIVE**.

Business - Couche Gold :

- Stockage des Data Marts finaux (Business Level Aggregates) sur une base de données conventionnelle (**PostgreSQL**) pour exposition aux équipes métiers et outils de BI.

Tables dans les parties Bronze, Silver et Gold :

Couche BRONZE (Données brutes - Tout en format PARQUET) :

- **bronze_yellow_tripdata :** Ingestion de notre fichier Parquet (déjà nativement en parquet : trajets février 2025).

- **bronze_taxi_zones :** Ingestion de notre référentiel géographique. Le flux source (CSV) sera lu par PySpark puis immédiatement sauvegardé au format Parquet dans HDFS pour respecter les standards d'optimisation Big Data.

- **bronze_data_dictionary** : Ingestion de notre table de correspondances métier (convertie et stockée en Parquet).

Couche SILVER (Data Warehouse sur HIVE - Format Parquet) :

- **silver_trips_enriched :** Table centrale propre, issue de la jointure entre les trajets (Dataset bronze n°1), les quartiers (Dataset bronze n°2) et les libellés métiers (Dataset bronze n°3). Les données aberrantes (par exemple : s'il y a des distances négatives) y sont filtrées.

Couche GOLD (Data Marts orientés métier sur PostgreSQL) :

- **gold_datamart_finance :** Agrégations (GROUP BY) des revenus, de la nouvelle taxe de congestion 2025 et des pourboires par quartier pour la Problématique 1.

- **gold_datamart_trafic :** Calcul de la durée des courses et de la vitesse moyenne pour repérer les zones de ralentissement et les bouchons (Problématique 2)..

- **gold_datamart_demande :** Comptage du nombre de trajets et de passagers selon le quartier de départ et d'arrivée, afin de savoir exactement où les taxis sont le plus demandés (Problématique 3).

4. API : Définition des endpoints

POST /api/v1/auth/login : Authentification de l'utilisateur et génération du token JWT.

archi : Dossiers
data/bronze
data/silver
data/gold

src/bronze
src/silver
src/gold
src/Analyse_EDA

endpoints pour accéder aux data marts :

GET /api/v1/finance/rentabilite : Problématique 1

GET /api/v1/trafic/fluidite : Problématique 2

GET /api/v1/demande/flux : Problématique 3

Pour garantir la sécurité et la stabilité de notre base de données Gold, toute requête envoyée à nos endpoints passera par un schéma de validation strict.

Exemple pour l'endpoint GET /api/v1/trafic/fluidite :

Le paramètre date doit obligatoirement être au format YYYY-MM-DD.

Le paramètre location_id doit obligatoirement être un nombre entier (Integer) compris entre 1 et 265 (correspondant aux ID valides de notre référentiel taxi_zones).
Toute requête ne respectant pas ce schéma sera rejetée avec une erreur "400 Bad Request" avant même d'atteindre notre base PostgreSQL.